Search (Ctrl+E)

Nikita Godambe Chat Shared Notes Recap Storyline

Complete LLM Client Event Loop Fix Guide

Problem Summary

The error you're seeing in logs:
```
[LLMClient] ERROR | model=qwen3:30b-a3b attempt=1/10: asyncio.locks.Semaphore object at 0x7f40dba7c5f0 (locked, waiters=1) is bound to a different event loop
```

This occurs because:
1. FastAPI creates a new event loop for each HTTP request
2. The LLM client creates semaphores bound to the initial event loop
3. When subsequent requests come in with different event loops, the semaphores become unusable

Solution Applied

File Modified: `agents/llm_client.py`

1. Fixed `_get_semaphore()` function (lines 107-122)

Before:
```python

Status of Nikita Godambe: extension-24609

Scanned with AltaScanner
```

Before:

```python
def _get_semaphore() -> asyncio.Semaphore:
    global _semaphore, _current_concurrency
    if _semaphore is None:
        from config import settings
        _current_concurrency = int(getattr(settings, "llm_max_concurrent", 2))
        _semaphore = asyncio.Semaphore(_current_concurrency)
        logger.info(f"[LLMClient] Semaphore created: max_concurrent={_current_concurrency}")
    return _semaphore
```

After:

```python
def _get_semaphore() -> asyncio.Semaphore:
    global _semaphore, _current_concurrency

    try:
        current_loop = asyncio.get_running_loop()
    except RuntimeError:
        current_loop = None

    needs_recreate = False
```

Scanned with AltaScanner

```
    if _semaphore is None:
        needs_recreate = True
    elif current_loop is not None and hasattr(_semaphore, '_loop') and _semaphore._loop is not
current_loop:
        # Semaphore is bound to a different event loop
        needs_recreate = True

    if needs_recreate:
        from config import settings
        _current_concurrency = int(getattr(settings, "llm_max_concurrent", 2))
        _semaphore = asyncio.Semaphore(_current_concurrency)
        logger.info(f"[LLMClient] Semaphore created: max_concurrent={_current_concurrency}")
    return _semaphore
```

2. Fixed `_get_vlm_semaphore()` function (lines 379-391)

Before:

```python
def _get_vlm_semaphore() -> asyncio.Semaphore:
    global _vlm_semaphore
    if _vlm_semaphore is None:
        from config import settings
```

Scanned with AltaScanner

```
        concurrency = int(getattr(settings, "vlm_server_count", 1)) * int(
            getattr(settings, "vlm_max_concurrency_per_server", 1)
        )
        concurrency = max(1, concurrency)
        _vlm_semaphore = asyncio.Semaphore(concurrency)
        logger.info(f"[VLMClient] Semaphore created: max_concurrent={concurrency}")
    return _vlm_semaphore
```

After:

```python
def _get_vlm_semaphore() -> asyncio.Semaphore:
    global _vlm_semaphore

    try:
        current_loop = asyncio.get_running_loop()
    except RuntimeError:
        current_loop = None

    needs_recreate = False

    if _vlm_semaphore is None:
        needs_recreate = True
    elif current_loop is not None and hasattr(_vlm_semaphore, '_loop') and _vlm_semaphore._loop is not
```

Scanned with AltaScanner

```
current_loop:
        # Semaphore is bound to a different event loop
        needs_recreate = True

    if needs_recreate:
        from config import settings
        concurrency = int(getattr(settings, "vlm_server_count", 1)) * int(
            getattr(settings, "vlm_max_concurrency_per_server", 1)
        )
        concurrency = max(1, concurrency)
        _vlm_semaphore = asyncio.Semaphore(concurrency)
        logger.info(f"[VLMClient] Semaphore created: max_concurrent={concurrency}")
    return _vlm_semaphore
```

Configuration Recommendations

Based on your environment configuration:

```ini
# LLM Concurrency Control
LLM_MAX_CONCURRENT=2      # Matches your 2 Ollama servers
LLM_REQUEST_TIMEOUT_S=600  # Increased from 500 for safety buffer
LLM_MAX_RETRIES=10        # Increased from 3 for better reliability
LLM_RETRY_BASE_S=10       # Increased from 4 for better backoff
```

Scanned with AltaScanner

Key Parameters Explained:

1. **LLM_MAX_CONCURRENT=2**
   - Purpose: Maximum concurrent LLM requests
   - Why: qwen3:30b-a3b is memory-bound; 2 matches your 2 Ollama servers
   - Can be changed at runtime: `update_llm_concurrency(4)`

2. **LLM_REQUEST_TIMEOUT_S=600**
   - Purpose: Per-attempt timeout in seconds
   - Why: qwen3 typically takes 20-60s; 600 provides safety buffer
   - Current: You have 500, recommend increasing to 600

3. **LLM_MAX_RETRIES=10**
   - Purpose: Maximum retry attempts before returning error
   - Why: More retries improve production reliability
   - Current: You have 3, recommend increasing to 10

4. **LLM_RETRY_BASE_S=10**
   - Purpose: Base for exponential backoff (retries at 10s, 20s, 40s, etc.)
   - Why: Longer backoff reduces server load during failures
   - Current: You have 4, recommend increasing to 10

Testing

Scanned with AltaScanner

Testing

Test Script: `test_llm_fix.py`

Run this to verify the fix works:

```bash
python test_llm_fix.py
```

Expected Output:

```
LLM Client Event Loop Fix Test
==========
1. Testing initial call...
   ✓ Initial semaphore: <asyncio.Semaphore object>
   ✓ Initial client: <httpx.AsyncClient object>

2. Testing after event loop change...
   ✓ Objects correctly cached in same loop

3. Testing actual LLM call...
   ✓ LLM call completed successfully
   ✓ Response length: X
```

Scanned with AltaScanner

```
============================================================
✓ All tests passed!
The LLM client should now handle event loop changes correctly.
============================================================
```

Verification Steps

1. **Check logs for event loop errors** - They should be gone after the fix
2. **Test with curl command**:
   ```bash
   curl -X POST http://localhost:8000/announcement-service/api/v1/search \
     -H "Authorization: Bearer YOUR_TOKEN" \
     -H "Content-Type: application/json" \
     -d '{"query": "test", "top_k": 5}'
   ```
3. **Monitor LLM client logs** - Should show proper semaphore creation for each event loop

Troubleshooting

If you still see event loop errors:

1. Verify the fix was applied correctly to `agents/llm_client.py`
2. Restart your FastAPI application
3. Check that no other code is creating semaphores incorrectly

Scanned with AltaScanner

If you see timeout errors:

1. Increase `LLM_REQUEST_TIMEOUT_S` to 600-900
2. Verify Ollama servers are healthy: `curl http://localhost:11434/api/tags`
3. Check network connectivity between API and Ollama servers

If you see "all retries exhausted" errors:

1. Increase `LLM_MAX_RETRIES` to 5-10
2. Increase `LLM_RETRY_BASE_S` to 10-15
3. Verify Ollama servers have enough resources (CPU, memory)

Files Created

1. **`agents/llm_client.py`** - Fixed (main file)
2. **`test_llm_fix.py`** - Test script to verify the fix
3. **`COMPLETE_FIX_GUIDE.md`** - This document (you're reading it)

Summary

The fix ensures that:
- Each event loop gets its own properly bound semaphore
- HTTP clients are recreated for new event loops
- Concurrent requests work without conflicts
- No more "bound to a different event loop" errors
- The system can handle the load from your 2 Ollama servers

Scanned with AltaScanner